Homelab Setup Guide — Demo-3 Three-Tier Edition

Run the Full Realistic Stack on Your Own Machine

DEFCON Coimbatore 2026 · Venugopal Parameswara

> *"The best way to understand resilience is to break things yourself. Then do it across four different tiers and see which one surprises you."*

What You'll Build

A fully functional B-V-R demonstration environment — five real Docker containers on an isolated network, a bash guardian polling every second, and a live browser dashboard showing per-tier MTTR in real time.

After this guide, you can:

- Run Demo-3 and measure your own R_1, R_2, R_3
- Execute all 6 Chaos GameDay cards
- Attempt the 4 CTF levels
- Extend guardian for your own scenarios

Total cost: Free | Requires ~4 GB RAM headroom and ~5 GB disk for images

---

Prerequisites

Minimum Hardware

- Any computer made after 2018 (needs to run 5 containers at once)
- 8 GB RAM minimum (16 GB recommended for a smooth demo)
- 10 GB free disk space
- macOS (Apple Silicon or Intel), Linux, or Windows 11 with WSL2

Software Stack

```
Docker Desktop (or Docker Engine on Linux)  — container runtime
docker compose v2                           — multi-container orchestration
Bash 5+                                     — guardian + attack scripts
Python 3.10+                                — ai_agent3.py + ms() in guardian
curl                                        — health checks + WAF attacks
psql client (optional)                      — DB verification from host
Chrome / Safari / Firefox                   — dashboard
tmux (recommended)                          — 4-pane demo layout
```

Installation by Platform

macOS (Apple Silicon — the reference platform)

```
# Install Homebrew if you don't have it
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HE

# Docker Desktop
brew install --cask docker
open -a Docker

# Bash 5 (macOS ships Bash 3 by default!)
brew install bash

# Tools
brew install tmux python@3.12 postgresql@15  # psql comes with postgresql

# Verify
docker --version          # 24.x or higher
/opt/homebrew/bin/bash --version   # 5.x
python3 --version          # 3.10+
```

Ubuntu / Debian Linux

```
sudo apt-get update
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker

sudo apt-get install -y tmux curl git bash python3 postgresql-client

docker --version
docker run --rm hello-world
```

Windows 11 with WSL2

```
# PowerShell as Administrator
wsl --install        # Installs WSL2 + Ubuntu
# Reboot when prompted
```

Then install Docker Desktop from docker.com, enable WSL integration for Ubuntu, and continue as Linux above inside the Ubuntu terminal.

Clone and Stage Demo-3

```
# The demo tree lives here — adapt paths if you're running elsewhere
mkdir -p ~/defcon-demo && cd ~/defcon-demo

# Extract the Demo-3 source (from the kit zip) into Demo-3/
unzip ~/Downloads/demo3_realistic.zip -d ~/defcon-demo/Demo-3

# Make scripts executable
cd ~/defcon-demo/Demo-3
chmod +x setup_and_run3.sh guardian/guardian3.sh attack3_*.sh

ls -la
# Should include: setup_and_run3.sh, docker-compose3.yml, ai_agent3.py,
#                 dashboard3.html, guardian/, web/, app/, db/, waf/, attack3_*.s
```

If you're running from a different absolute path, you must edit two files:

```
# ai_agent3.py line 17
DEMO3_DIR = "/Users/kuttanadan/Documents/defcon-demo/Demo-3"

# guardian/guardian3.sh line 19
DEMO3_DIR="/Users/kuttanadan/Documents/defcon-demo/Demo-3"
```

Change both to your actual path, then save.

First Run — Full Stack

Build All Images (one-time, 10–15 minutes)

```
cd ~/defcon-demo/Demo-3
./setup_and_run3.sh setup
```

You'll see five builds run sequentially:

```
Building demo3-web:golden       (nginx:alpine + html + curl)
Building demo3-app:golden       (python:3.11-alpine + flask + psycopg2)
Building demo3-db-primary:golden (postgres:15-alpine + init-primary.sh)
Building demo3-db-replica:golden (postgres:15-alpine + setup-replica.sh)
Building demo3-waf:golden       (owasp/modsecurity-crs:nginx-alpine)
```

Start the Stack

```
./setup_and_run3.sh start
```

This brings up the 5 containers, starts `guardian3.sh` in the background (or foreground — depends on your launch mode), starts `ai_agent3.py` on port 7880, and opens the dashboard.

Verify All Five Are Healthy

```
docker ps --filter "name=demo3-" --format "table {{.Names}}\t{{.Status}}"
```

Expected:

```
NAMES            STATUS
demo3-web        Up 30 seconds (healthy)
demo3-app        Up 28 seconds (healthy)
demo3-db-pri     Up 40 seconds (healthy)
demo3-db-rep     Up 35 seconds (healthy)
demo3-waf        Up 30 seconds (healthy)
```

All five must say `(healthy)`. If any are `(unhealthy)` or `(starting)`, check logs:

```
docker logs demo3-<name> --tail 50
```

The Demo Layout — tmux

A 4-pane tmux layout gives the audience the right visual. Pane 1 is the guardian, pane 2 is the attacker terminal, pane 3 is psql on the replica, pane 4 is the browser (or a curl-loop).

```
tmux new-session -s demo3
tmux split-window -h
tmux split-window -v
tmux select-pane -t 0
tmux split-window -v

# Pane 0 (top-left): Guardian
cd ~/defcon-demo/Demo-3 && tail -f /tmp/demo3_guardian.jsonl | \
  python3 -c "
import sys, json
for line in sys.stdin:
    try: d=json.loads(line); print(f'{d[\"ts\"]} [{d[\"tier\"]:7}] {d[\"event\"]
    except: pass
"

# Pane 1 (top-right): Attacker terminal — empty, ready for attack3_*.sh

# Pane 2 (bottom-left): Live replication status
watch -n 1 "docker exec demo3-db-pri psql -U phoenix -d phoenix -t -c \
  'SELECT state, pg_wal_lsn_diff(sent_lsn, replay_lsn) AS lag FROM pg_stat_repli

# Pane 3 (bottom-right): Open http://localhost:7880 in the browser
```

Running the Demo — The Four Attacks

Open http://localhost:7880 in an incognito window (incognito avoids JS cache issues).

Attack 1 — Web Tier Wipeout (R ≈ 0.63s)

```
./setup_and_run3.sh attack web
```

Watch the dashboard: the WEB ring goes red → amber → green within 1 second. The Event Log shows `DRIFT_DETECTED → RESPAWN_START → RESPAWN_SUCCESS mttr_s=0.63`.

Attack 2 — App Tier Cascade (R ≈ 0.73s)

```
./setup_and_run3.sh attack app
```

Dashboard: APP ring red → amber → green. Check the APP card — **session count dropped to 0, transaction count unchanged.** That's the point.

Attack 3 — DB Ransomware (R ≈ 12.56s)

```
./setup_and_run3.sh attack db
```

Dashboard: DB ring pulses red as writes spike, then enters RECOVERING state while replica is fenced. After 10 quiet ticks (~10s), it flips green with `REPLICA_RESUMED mttr_s=12.50`.

Attack 4 — WAF Gauntlet (R = 0, 13/15 blocked)

```
./setup_and_run3.sh attack waf
```

The WAF panel (may be hidden until clicked the first time) populates with a live attacker-vs-defender split view. Real 403s appear in the ModSecurity audit log:

```
docker exec demo3-waf tail -5 /var/log/modsec/audit.log
```

Five Self-Paced Exercises

Work through these in order.

Exercise 1 — Baseline Per-Tier MTTR

```
for i in 1 2 3; do
  ./setup_and_run3.sh attack web
  sleep 3
done

grep RESPAWN_SUCCESS /tmp/demo3_guardian.jsonl | tail -3
# Record the mttr_ms for each run; compute average.
```

Do the same for `app` and `db`. You should converge on:

- WEB: ~600–700 ms
- APP: ~700–800 ms
- DB: ~12,000–13,000 ms

Question: Is your WEB R under 1 second? If not, what's slowing it down? (First-run image pull? Docker Desktop CPU throttling? Container restart policy?)

Exercise 2 — State Preservation Proof

```
# Before the attack
curl -s localhost:3001/status | python3 -m json.tool

# Note the transactions and sessions_active values.

./setup_and_run3.sh attack app
sleep 5

# After the attack
curl -s localhost:3001/status | python3 -m json.tool
```

Question: Did the transaction count change? Did the session count change? Why did one survive and the other not?

Exercise 3 — Quantify the DB Leak

Run the CTF Level 1 procedure. Reproduce the ~400–500 row leak number on your own hardware. Submit to a team member as a Level 1 flag.

Exercise 4 — Watch the WAF Audit Log Live

```
docker exec demo3-waf tail -f /var/log/modsec/audit.log
```

In another terminal: `./setup_and_run3.sh attack waf`

Question: Can you map each of the 15 attack-script waves to a ModSecurity rule ID in the audit log? The talk references rules 942100 (SQLi), 941100 (XSS), 930100 (path traversal). Find the exact rule ID for each wave.

Exercise 5 — Extend Guardian

Pick one of the documented §9 gaps and close it:

- Add file-checksum drift detection to WEB (Demo-2 CTF Level 3 — still unfixed in Demo-3)
- Tighten the DB fence trigger from 500 rows/sec to 200
- Add replication-slot health monitoring

Success criteria: your patched guardian catches the scenario *and* MTTR does not regress.

MTTR Measurement Tool

Drop this into `~/defcon-demo/Demo-3/measure-mttr.sh` and `chmod +x` it.

```
#!/usr/bin/env bash
# measure-mttr.sh — Run N attacks on a target tier and report statistics.
# Usage: ./measure-mttr.sh web 5      ← 5 web attacks
#        ./measure-mttr.sh app 3
#        ./measure-mttr.sh db 2

TARGET=${1:-web}
N=${2:-5}

case "$TARGET" in
  web) EVENT="RESPAWN_SUCCESS" ;;
  app) EVENT="RESTART_SUCCESS" ;;
  db)  EVENT="REPLICA_RESUMED" ;;
  *)   echo "Unknown target: $TARGET"; exit 1 ;;
esac

echo "Running $N attacks on tier: $TARGET"
RESULTS=()

for i in $(seq 1 $N); do
  BEFORE=$(grep -c "$EVENT" /tmp/demo3_guardian.jsonl 2>/dev/null || echo 0)
  ./setup_and_run3.sh attack "$TARGET" >/dev/null 2>&1 || true

  # Wait for the next success event to appear
  while true; do
    AFTER=$(grep -c "$EVENT" /tmp/demo3_guardian.jsonl 2>/dev/null || echo 0)
    [[ "$AFTER" -gt "$BEFORE" ]] && break
    sleep 0.5
  done

  MTTR=$(grep "$EVENT" /tmp/demo3_guardian.jsonl | tail -1 | \
         python3 -c "import sys,json; print(json.loads(sys.stdin.read())['mttr_m
  RESULTS+=($MTTR)
  echo "  Attack $i: ${MTTR}ms"
  sleep 4
done

# Stats
python3 -c "
r = [${RESULTS[@]/%/,}]; r = [x for x in r if x]
if r:
  print(f'  ──────────────────────────────')
  print(f'  MTTR for tier \"$TARGET\" (n={len(r)})')
  print(f'  avg: {sum(r)//len(r)} ms')
  print(f'  min: {min(r)} ms')
  print(f'  max: {max(r)} ms')
"
```

Troubleshooting

Docker permission denied

```
sudo usermod -aG docker $USER
newgrp docker
```

Port 9091 / 3001 / 5432 / 5433 / 8090 / 7880 already in use

```
lsof -iTCP:9091 -sTCP:LISTEN
# Kill the offending process or edit docker-compose3.yml to change host ports
```

`(unhealthy)` on `demo3-web` despite HTTP 200

This is the §10 issue-C from the technical doc. The compose file healthcheck used `wget` (busybox wget resolves `localhost` to `::1`, and nginx doesn't listen on IPv6). The fix in this codebase is `curl -fs http://127.0.0.1/health`. If you see this: rebuild with `./setup_and_run3.sh setup` against an updated compose file.

Dashboard stuck on "CONNECTING..."

This is §10 issue-A — `drawSparkline` throwing a `SyntaxError` inside `runPoll`. The fix is in `dashboard3.html` (explicit `SPARK_RGBA` map instead of CSS-var string-mangling). If you're running an older build, hard-refresh (Cmd+Shift+R) or pull the latest source.

WAF panel shows `REQUESTS SENT = 0` after an attack

The panel is hidden until the WAF button is clicked once. Click it before attacking.

Replication lag grows unboundedly

```
docker exec demo3-db-pri psql -U phoenix -d phoenix -c \
  "SELECT * FROM pg_stat_replication;"
```

If `state != streaming`, the replica has disconnected. Restart it:

```
docker restart demo3-db-rep
```

Guardian exits immediately

```
bash --version           # Need 5+
/opt/homebrew/bin/bash /Users/kuttanadan/Documents/defcon-demo/Demo-3/guardian/g
```

(macOS system Bash is 3.2 — homebrew Bash works.)

DB attack doesn't trigger MASS_WRITE_DETECTED

The baseline counter in guardian is `db_write_baseline`. On first run it's 0, which means the first tick after startup establishes a baseline *above* 500 purely from guardian's first read. Restart guardian after the first `setup_and_run3.sh start` if you see this.

Going Further — Production Patterns

Homelab pattern	Production equivalent
`guardian3.sh` bash loop	Kubernetes operator reconcile loop (operator-sdk / kubebuilder)
`docker rm -f` + `docker run`	`kubectl rollout restart deployment/<name>`
Golden image tag `:golden`	OCI image with cosign signature + SBOM attestation
`pg_wal_replay_pause()` fence	Patroni paused replication + DCS-mediated failover
ModSecurity nginx	Envoy filter chain with the ModSecurity WASM plugin, or F5 NGINX App Protect
`/tmp/demo3_guardian.jsonl`	Loki / Elasticsearch / CloudWatch Logs, with a retention policy
`ai_agent3.py` dashboard	Grafana + Prometheus, or Honeycomb traces keyed by `mttr_ms`
Single-host deployment	3 replicas per tier across 3 AZs, anti-affinity rules

---

DEFCON Coimbatore 2026 · Demo-3 · Venugopal Parameswara github.com/CyberRant-lab/Defcon